

TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline – Customer Order Processing

-

1. Document Overview

Purpose

This Technical Requirements Document translates the Functional Requirements Document into implementation-ready technical specifications for an AWS Glue-based data pipeline. It defines the technical architecture, data structures, transformation logic, and runtime behavior required to implement customer and order data processing with SCD Type 2 tracking using Apache Hudi.

Scope

This TRD covers:

- Technical specifications for ingesting CSV files from S3 using AWS Glue
- Data quality transformation logic using PySpark
- AWS Glue Data Catalog table registration specifications
- Apache Hudi configuration for SCD Type 2 implementation
- Incremental processing logic for customer dimension changes
- Aggregation logic and output specifications
- Schema definitions for all source and target entities
- Error handling and logging requirements
- Runtime parameter specifications

This TRD does NOT cover:

- AWS IAM role and policy JSON definitions
- CloudWatch alarm configuration details
- Lake Formation permission grants
- Network or VPC configurations
- Cost optimization strategies
- Downstream consumer application integration

Assumptions

- AWS Glue version 3.0 or higher with Spark 3.1+ and Python 3.7+
- Apache Hudi libraries version 0.10.0 or higher available in Glue job

- CSV files use comma delimiter with header row
- CSV files are UTF-8 encoded
- "Null" string literal (case-insensitive) and empty/NULL values are treated as invalid
- Date field format in Order data is parseable by Spark's date parsing functions (ISO 8601 assumed)
- PricePerUnit and Qty are numeric types (Double/Decimal)
- Customer changes are detected by comparing Name, EmailId, or Region attributes
- CustId is the primary key for Customer entity
- OrderId is the primary key for Order entity
- S3 bucket `adif-sdlc` and all specified paths exist with appropriate read/write permissions
- Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` exists
- Glue job has permissions to read from source S3 paths and write to target S3 paths
- Glue job has permissions to create/update Glue Data Catalog tables
- For SCD Type 2, OpTs (operation timestamp) uses job execution timestamp
- For first-time load, all records in ordersummary are inserted with IsActive=True, StartDate=current_timestamp, EndDate=NULL
- Hudi record key for ordersummary is OrderId
- Hudi precombine key for ordersummary is OpTs
- TotalAmount calculation: PricePerUnit × Qty (no rounding specified, use default precision)
- Customer aggregate spend groups by customer Name and order Date (not CustId)
- No schema evolution is expected during pipeline execution
- Duplicate detection is based on exact match across all columns
- No data encryption requirements specified beyond default S3 encryption
-

2. Technical Requirements

TR-INGEST-001: Technical Implementation for Customer Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 source path and load into Spark DataFrame with defined schema for downstream processing.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: `s3://adif-sdlc/sdlc\_wizard/customerdata/`
- File Format: CSV

- Delimiter: Comma (,)
- Header: Yes (first row)
- Encoding: UTF-8
- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: `customer\_df`
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:
 1. Use `spark.read.format("csv")` with options:
 - `header=true`
 - `inferSchema=false` (explicit schema definition)
 - `delimiter=","`
 - `encoding="UTF-8"`
 2. Define explicit schema using StructType with fields: CustId (StringType), Name (StringType), EmailId (StringType), Region (StringType)
 3. Apply schema during read operation
 4. Load all CSV files matching the path pattern into single DataFrame
 5. No transformations applied at this stage
- Validation Rules:
 - Verify S3 path exists and is accessible
 - Verify at least one CSV file exists in the path
 - Verify CSV structure matches expected 4-column schema
 - Verify header row contains expected column names (case-insensitive match acceptable)
- Error Handling and Logging:
 - If S3 path does not exist or is inaccessible: Log error with path details, raise exception, fail job

- If no CSV files found: Log error "No CSV files found in path", raise exception, fail job
- If CSV structure mismatch: Log error with file name and schema mismatch details, raise exception, fail job
- If read operation fails: Log exception stack trace, fail job
- Log INFO: "Starting customer data ingestion from {path}"
- Log INFO: "Successfully loaded {row_count} customer records"
- Runtime Parameters:
 - `SOURCE\_CUSTOMER\_PATH`: S3 path for customer CSV files (default: `s3://adif-sdlc/sdlc\_wizard/customerdata/`)
- Operational Notes:
 - Use Glue DynamicFrame or Spark DataFrame based on implementation preference
 - Recommend Spark DataFrame for explicit schema control
 - No caching required at this stage unless memory optimization needed
-

TR-INGEST-002: Technical Implementation for Order Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order CSV files from S3 source path and load into Spark DataFrame with defined schema for downstream processing.

- Source Details:
 - Source Type: S3 CSV Files
 - Source Path: `s3://adif-sdlc/sdlc\_wizard/orderdata/`
 - File Format: CSV
 - Delimiter: Comma (,)
 - Header: Yes (first row)
 - Encoding: UTF-8
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: `order\_df`
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	

	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:

1. Use ``spark.read.format("csv")`` with options:

- ``header=true``
- ``inferSchema=false`` (explicit schema definition)
- ``delimiter=","``
- ``encoding="UTF-8"``

2. Define explicit schema using StructType with fields: OrderId (StringType), ItemName (StringType), PricePerUnit (DoubleType), Qty (DoubleType), Date (StringType), CustId (StringType)

3. Apply schema during read operation

4. Load all CSV files matching the path pattern into single DataFrame

5. No transformations applied at this stage

- Validation Rules:

- Verify S3 path exists and is accessible
- Verify at least one CSV file exists in the path
- Verify CSV structure matches expected 6-column schema
- Verify header row contains expected column names (case-insensitive match acceptable)
- Verify PricePerUnit and Qty can be cast to numeric types

- Error Handling and Logging:

- If S3 path does not exist or is inaccessible: Log error with path details, raise exception, fail job
- If no CSV files found: Log error "No CSV files found in path", raise exception, fail job
- If CSV structure mismatch: Log error with file name and schema mismatch details, raise exception, fail job
- If numeric type casting fails: Log error with row details, raise exception, fail job
- If read operation fails: Log exception stack trace, fail job
- Log INFO: "Starting order data ingestion from {path}"
- Log INFO: "Successfully loaded {row_count} order records"

- Runtime Parameters:
- `SOURCE\_ORDER\_PATH`: S3 path for order CSV files (default: `s3://adif-sdlc/sdlc\_wizard/orderdata/`)
- Operational Notes:
- Use Glue DynamicFrame or Spark DataFrame based on implementation preference
- Recommend Spark DataFrame for explicit schema control
- No caching required at this stage unless memory optimization needed
-

TR-CLEAN-001: Technical Implementation for Data Quality Cleaning

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove rows containing "Null" string literal or NULL/empty values, and eliminate duplicate records from customer and order datasets.

- Source Details:
- Source Type: In-memory Spark DataFrames
- Source Names: `customer\_df` (from TR-INGEST-001), `order\_df` (from TR-INGEST-002)
- Target Details:
- Target Type: In-memory Spark DataFrames
- Target Names: `customer\_cleaned\_df`, `order\_cleaned\_df`
- Input Schema:
- Customer: Same as TR-INGEST-001 output schema
- Order: Same as TR-INGEST-002 output schema
- Output Schema:
- Customer: Same as input schema (cleaned subset)
- Order: Same as input schema (cleaned subset)
- Processing / Transformation Logic:

For Customer DataFrame:

1. Filter out rows where any column contains string "Null" (case-insensitive):
- Use `lower(col\_name).contains("null")` for each column
 - Combine conditions with OR logic
 - Negate the condition to keep valid rows

2. Filter out rows where any column is NULL or empty:

- Use <code>`col_name.isNull()</code>	<code>(col_name == "")`</code> for each column
---------------------------------------	--

- Combine conditions with OR logic
- Negate the condition to keep valid rows

3. Remove duplicate rows:

- Use ``dropDuplicates()`` without parameters (considers all columns)

4. Assign result to ``customer_cleaned_df``

For Order DataFrame:

1. Filter out rows where any column contains string "Null" (case-insensitive):

- Use ``lower(col_name).contains("null")`` for each column
- Combine conditions with OR logic
- Negate the condition to keep valid rows

2. Filter out rows where any column is NULL or empty:

- Use <code>`col_name.isNull()</code>	<code>(col_name == "")`</code> for each column
---------------------------------------	--

- Combine conditions with OR logic
- Negate the condition to keep valid rows

3. Remove duplicate rows:

- Use ``dropDuplicates()`` without parameters (considers all columns)

4. Assign result to ``order_cleaned_df``

• Validation Rules:

- Count records before and after cleaning for each dataset
- Log number of records removed due to null values
- Log number of duplicate records removed
- Verify cleaned DataFrames are not empty (warning if empty, not failure)

• Error Handling and Logging:

- If cleaning operation fails: Log exception stack trace, fail job
- If all records removed: Log WARNING "All records removed during cleaning for {dataset_name}", continue processing
- Log INFO: "Customer cleaning: {original_count} -> {cleaned_count} records ({removed_count} removed)"
- Log INFO: "Order cleaning: {original_count} -> {cleaned_count} records ({removed_count} removed)"
- Log INFO: "Customer duplicates removed: {duplicate_count}"
- Log INFO: "Order duplicates removed: {duplicate_count}"

- Runtime Parameters:

None

- Operational Notes:
- Null string matching should be case-insensitive
- Empty string check applies only to String type columns
- Duplicate detection is exact match across all columns
- Consider caching cleaned DataFrames if used multiple times downstream
-

TR-CATALOG-001: Technical Implementation for Glue Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer and order datasets to S3 in Parquet format and register as queryable tables in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Spark DataFrames
- Source Names: `customer\_cleaned\_df` (from TR-CLEAN-001), `order\_cleaned\_df` (from TR-CLEAN-001)
- Target Details:

Customer Target:

- Target Type: AWS Glue Data Catalog Table
- Database: `gen\_ai\_poc\_databrickscoe`
- Table Name: `sdlc\_wizard\_customer`
- S3 Location: `s3://adif-sdlc/catalog/sdlc\_wizard/customer/`
- File Format: Parquet

Order Target:

- Target Type: AWS Glue Data Catalog Table
- Database: `gen\_ai\_poc\_databrickscoe`
- Table Name: `sdlc\_wizard\_order`
- S3 Location: `s3://adif-sdlc/catalog/sdlc\_wizard/order/`
- File Format: Parquet
- Input Schema:

- Customer: Same as TR-CLEAN-001 output schema for customer
- Order: Same as TR-CLEAN-001 output schema for order
- Output Schema:

Customer Table:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

Order Table:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Processing / Transformation Logic:

For Customer:

1. Write `customer\_cleaned\_df` to S3 using `write.format("parquet")`
2. Use `mode("overwrite")` to replace existing data
3. Write to path: `s3://adif-sdlc/catalog/sdlc\_wizard/customer/`
4. Register table in Glue Data Catalog:
 - Use `glueContext.create\_data\_frame.from\_catalog()` or direct catalog API
 - Database: `gen\_ai\_poc\_databrickscoe`
 - Table: `sdlc\_wizard\_customer`
 - Location: S3 path from step 3
 - Format: Parquet
 - Schema: Infer from DataFrame or specify explicitly

For Order:

1. Write `order\_cleaned\_df` to S3 using `write.format("parquet")`

2. Use ``mode("overwrite")`` to replace existing data
3. Write to path: ``s3://adif-sdlc/catalog/sdlc_wizard/order/``
4. Register table in Glue Data Catalog:
 - Use ``glueContext.create_data_frame.from_catalog()`` or direct catalog API
 - Database: ``gen_ai_poc_databrickscoe``
 - Table: ``sdlc_wizard_order``
 - Location: S3 path from step 3
 - Format: Parquet
 - Schema: Infer from DataFrame or specify explicitly
 - Validation Rules:
 - Verify database ``gen_ai_poc_databrickscoe`` exists before table creation
 - Verify S3 write completes successfully
 - Verify Glue catalog table creation/update completes successfully
 - Verify table is queryable via Athena (optional validation query)
 - Error Handling and Logging:
 - If database does not exist: Log error "Database {database_name} not found", raise exception, fail job
 - If S3 write fails: Log error with S3 path and exception details, raise exception, fail job
 - If catalog registration fails: Log error with table name and exception details, raise exception, fail job
 - Log INFO: "Writing customer data to {s3_path}"
 - Log INFO: "Registered Glue catalog table {database}.{table}"
 - Log INFO: "Writing order data to {s3_path}"
 - Log INFO: "Registered Glue catalog table {database}.{table}"
 - Runtime Parameters:
 - ``CATALOG_DATABASE``: Glue catalog database name (default: ``gen_ai_poc_databrickscoe``)
 - ``CATALOG_CUSTOMER_TABLE``: Customer table name (default: ``sdlc_wizard_customer``)
 - ``CATALOG_ORDER_TABLE``: Order table name (default: ``sdlc_wizard_order``)
 - ``CATALOG_CUSTOMER_PATH``: S3 path for customer Parquet files (default: ``s3://adif-sdlc/catalog/sdlc_wizard/customer/``)
 - ``CATALOG_ORDER_PATH``: S3 path for order Parquet files (default: ``s3://adif-sdlc/catalog/sdlc_wizard/order/``)
 - Operational Notes:
 - Use overwrite mode to ensure clean state on each run

- Parquet compression codec: Use default (Snappy) unless specified otherwise
- No partitioning specified for these tables
- Catalog registration can use Glue API or Spark SQL CREATE TABLE statement
- Ensure Glue job has `glue:CreateTable` and `glue:UpdateTable` permissions
-

TR-SCD2-001: Technical Implementation for SCD Type 2 Order Summary

- Related Functional Requirement ID(s): FR-SCD2-001, FR-INCR-001
- Objective:

Join customer and order data, implement SCD Type 2 logic to track historical changes in customer attributes, and store result in Apache Hudi format with Glue catalog registration.

- Source Details:
 - Source Type: In-memory Spark DataFrames
 - Source Names:
 - `customer\_cleaned\_df` (from TR-CLEAN-001)
 - `order\_cleaned\_df` (from TR-CLEAN-001)
 - Existing Hudi table `ordersummary` (for incremental updates)
- Target Details:
 - Target Type: Apache Hudi Table (Copy-on-Write)
 - Target Name: `ordersummary`
 - S3 Location: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
 - Glue Catalog Database: `gen\_ai\_poc\_databrickscoe`
 - Glue Catalog Table: `ordersummary`
 - Hudi Table Type: COPY_ON_WRITE
 - Hudi Record Key: OrderId
 - Hudi Precombine Key: OpTs
 - Hudi Partition Path: None (non-partitioned)
- Input Schema:

Customer (from TR-CLEAN-001):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	

	EmailId	String	Variable	No	
	Region	String	Variable	No	

Order (from TR-CLEAN-001):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

• Output Schema:

Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
-----	-----	-----	-----	-----
OrderId	String	Variable	No	Direct map from Order.OrderId
ItemName	String	Variable	No	Direct map from Order.ItemName
PricePerUnit	Double	Double precision	No	Direct map from Order.PricePerUnit
Qty	Double	Double precision	No	Direct map from Order.Qty
Date	String	Variable	No	Direct map from Order.Date
CustId	String	Variable	No	Direct map from Order.CustId
Name	String	Variable	No	Direct map from Customer.Name
EmailId	String	Variable	No	Direct map from Customer.EmailId
Region	String	Variable	No	Direct map from Customer.Region
IsActive	Boolean	Boolean	No	Derived: True for current record, False for historical
StartDate	Timestamp	Timestamp	No	Derived: Job execution timestamp for new/updated records
EndDate	Timestamp	Timestamp	Yes	Derived: NULL for active records, job execution timestamp for closed records
OpTs	Timestamp	Timestamp	No	Derived: Job execution timestamp (used as precombine key)

• Processing / Transformation Logic:

Initial Load (when Hudi table does not exist):

1. Join `customer\_cleaned\_df` and `order\_cleaned\_df` on `CustId` using inner join
2. Add SCD Type 2 columns:
 - `IsActive = True`
 - `StartDate = current\_timestamp()`
 - `EndDate = NULL`
 - `OpTs = current\_timestamp()`

3. Write to Hudi table using upsert operation with configuration:

- ``hoodie.table.name = ordersummary``
- ``hoodie.datasource.write.recordkey.field = OrderId``
- ``hoodie.datasource.write.precombine.field = OpTs``
- ``hoodie.datasource.write.operation = upsert``
- ``hoodie.datasource.write.table.type = COPY_ON_WRITE``
- ``hoodie.datasource.write.hive_style_partitioning = false``

4. Register table in Glue Data Catalog

Incremental Update (when Hudi table exists):

1. Read existing Hudi table into DataFrame ``existing_ordersummary_df``
2. Join ``customer_cleaned_df`` and ``order_cleaned_df`` on ``CustId`` to create ``new_ordersummary_df``
3. Add temporary columns to ``new_ordersummary_df``: ``IsActive = True``, ``StartDate = current_timestamp()``, ``EndDate = NULL``, ``OpTs = current_timestamp()``
4. Identify changed customer records:
 - Join ``existing_ordersummary_df`` (filter ``IsActive = True``) with ``new_ordersummary_df`` on ``OrderId``
 - Compare Name, EmailId, Region between existing and new
 - Flag records where any of these attributes differ as "changed"
5. For changed records:
 - Create "close" records: Copy from existing with ``IsActive = False``, ``EndDate = current_timestamp()``, ``OpTs = current_timestamp()``
 - Create "new" records: Use new customer attributes with ``IsActive = True``, ``StartDate = current_timestamp()``, ``EndDate = NULL``, ``OpTs = current_timestamp()``
6. For unchanged records:
 - Keep existing active records as-is (no update needed)
7. For new orders (OrderId not in existing table):
 - Insert with ``IsActive = True``, ``StartDate = current_timestamp()``, ``EndDate = NULL``, ``OpTs = current_timestamp()``
8. Combine close records and new records into single DataFrame
9. Write to Hudi table using upsert operation (same configuration as initial load)
 - Validation Rules:
 - Verify join produces at least one record (log warning if empty)
 - Verify OrderId is unique in the joined dataset before Hudi write
 - Verify Hudi record key (OrderId) is not null
 - Verify OpTs is populated for all records

- Verify only one active record exists per OrderId after upsert
- Error Handling and Logging:
 - If join produces no records: Log WARNING "No records produced from customer-order join", skip Hudi write, continue job
 - If Hudi write fails: Log error with exception details, raise exception, fail job
 - If catalog registration fails: Log error, raise exception, fail job
 - If customer has no matching orders: No ordersummary record created (expected behavior, log INFO)
 - Log INFO: "Joined customer and order data: {record_count} records"
 - Log INFO: "Identified {changed_count} changed customer records"
 - Log INFO: "Writing {record_count} records to Hudi table ordersummary"
 - Log INFO: "Hudi upsert completed successfully"
 - Log INFO: "Registered Glue catalog table {database}.{table}"
- Runtime Parameters:
 - ``CURATED_ORDERSUMMARY_PATH``: S3 path for ordersummary Hudi table (default: ``s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``)
 - ``CATALOG_DATABASE``: Glue catalog database name (default: ``gen_ai_poc_databrickscoe``)
 - ``CATALOG_ORDERSUMMARY_TABLE``: Order summary table name (default: ``ordersummary``)
 - ``HUDI_TABLE_NAME``: Hudi table name (default: ``ordersummary``)
- Operational Notes:
 - Use Hudi COPY_ON_WRITE table type for read-optimized performance
 - Hudi precombine key (OpTs) ensures latest record wins in case of conflicts
 - For incremental logic, detect changes by comparing customer attributes (Name, EmailId, Region)
 - Use Spark SQL window functions or self-join to identify changed records
 - Ensure Hudi libraries are available in Glue job classpath
 - Hudi configuration should be passed via DataFrame write options
 - Catalog registration can use Hudi's built-in sync or manual Glue API call
 - Consider enabling Hudi metadata table for large datasets
 - No partitioning specified; use non-partitioned table
-

TR-AGG-001: Technical Implementation for Customer Aggregate Spend

- Related Functional Requirement ID(s): FR-AGG-001

- Objective:

Aggregate order data by customer name and date to calculate total spend, write to S3 in Parquet format, and register in Glue Data Catalog.

- Source Details:

- Source Type: In-memory Spark DataFrames

- Source Names:

- `customer\_cleaned\_df` (from TR-CLEAN-001)

- `order\_cleaned\_df` (from TR-CLEAN-001)

- Target Details:

- Target Type: AWS Glue Data Catalog Table

- Database: `gen\_ai\_poc\_databrickscoe`

- Table Name: `customeragggregatespend`

- S3 Location: `s3://adif-sdlc/analytics/customeragggregatespend/`

- File Format: Parquet

- Input Schema:

Customer (from TR-CLEAN-001):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

Order (from TR-CLEAN-001):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	Name	String	Variable	No	Direct map from Customer.Name	
	TotalAmount	Double	Double precision	No	Derived: SUM(Order.PricePerUnit × Order.Qty)	
	Date	String	Variable	No	Direct map from Order.Date	

- Processing / Transformation Logic:

1. Join `customer\_cleaned\_df` and `order\_cleaned\_df` on `CustId` using inner join
2. Create calculated column `TotalAmount = PricePerUnit Qty` for each row
3. Group by `Name` (from Customer) and `Date` (from Order)
4. Aggregate: `SUM(TotalAmount)` as `TotalAmount`
5. Select final columns: `Name`, `TotalAmount`, `Date`
6. Write result to S3 using `write.format("parquet")`
7. Use `mode("overwrite")` to replace existing data
8. Write to path: `s3://adif-sdlc/analytics/customeraggregatespend/`

9. Register table in Glue Data Catalog:

- Database: `gen\_ai\_poc\_databrickscoe`
- Table: `customeraggregatespend`
- Location: S3 path from step 8
- Format: Parquet
- Schema: Name (String), TotalAmount (Double), Date (String)

- Validation Rules:

- Verify join produces at least one record (create empty table with schema if no records)
- Verify TotalAmount calculation does not produce NULL values
- Verify aggregation completes successfully
- Verify S3 write completes successfully
- Verify Glue catalog registration completes successfully

- Error Handling and Logging:

- If join produces no records: Log WARNING "No records produced from customer-order join for aggregation", create empty Parquet file with schema, register table, continue job
- If aggregation fails: Log error with exception details, raise exception, fail job
- If S3 write fails: Log error with S3 path and exception details, raise exception, fail job
- If catalog registration fails: Log error with table name and exception details, raise exception, fail job
- Log INFO: "Joined customer and order data for aggregation: {record_count} records"
- Log INFO: "Aggregated data by Name and Date: {aggregated_count} records"

- Log INFO: "Writing customer aggregate spend to {s3_path}"
- Log INFO: "Registered Glue catalog table {database}.{table}"
- Runtime Parameters:
 - `ANALYTICS\_CUSTOMERAGGREGATESPEND\_PATH`: S3 path for customer aggregate spend Parquet files (default: `s3://adif-sdlc/analytics/customeragggregatespend/`)
 - `CATALOG\_DATABASE`: Glue catalog database name (default: `gen\_ai\_poc\_databrickscoe`)
 - `CATALOG\_CUSTOMERAGGREGATESPEND\_TABLE`: Customer aggregate spend table name (default: `customeragggregatespend`)
- Operational Notes:
 - Use overwrite mode to ensure clean state on each run
 - TotalAmount calculation uses default double precision (no rounding specified)
 - Grouping is by customer Name (not CustId) as specified in FRD
 - No partitioning specified for this table
 - Parquet compression codec: Use default (Snappy) unless specified otherwise
 - Ensure Glue job has `glue:CreateTable` and `glue:UpdateTable` permissions
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: Python (PySpark)
- Glue Version: AWS Glue 3.0 or higher
- Spark Version: 3.1 or higher
- Python Version: 3.7 or higher

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Execution Mode: Standard (not streaming)
- Trigger: On-demand or scheduled (not specified in FRD)
- Concurrency: Single job execution (no parallel runs specified)
- Retry Strategy: Use Glue job retry configuration (default: 0 retries, can be configured)
- Timeout: Configure based on data volume (recommend starting with 60 minutes)

Glue Job Type Expectations

- Worker Type: G.1X or G.2X (recommend G.1X for initial implementation)
- Number of Workers: Start with 2-5 workers, scale based on data volume
- Max Capacity: Not applicable (using worker type configuration)

- Job Bookmark: Enable for incremental processing of source files (optional)
- Continuous Logging: Enable for real-time CloudWatch log streaming
- Spark UI: Enable for debugging and performance tuning
- Metrics: Enable CloudWatch metrics for job monitoring
- Security Configuration: Use if encryption at rest/in transit is required
- Connections: None required (S3 access via IAM role)
- Additional Libraries: Apache Hudi JAR files (version 0.10.0 or higher)
- Python Libraries: None beyond standard Glue/Spark libraries
- Script Location: Store in S3 bucket with versioning enabled
-

4. Data Design Details

Source Paths / Source Systems

	Source Entity	Source Type	Source Path	Format	Schema Defined In	
	-----	-----	-----	-----	-----	
	Customer	S3 CSV	`s3://adif-sdlc/sdlc_wizard/customerdata/`	CSV	TR-INGEST-001	
	Order	S3 CSV	`s3://adif-sdlc/sdlc_wizard/orderdata/`	CSV	TR-INGEST-002	

Target Paths / Target Tables

Table Type	S3 Path	Glue Database	Glue Table	Format
-----	-----	-----	-----	-----
Table	`s3://adif-sdlc/catalog/sdlc_wizard/customer/`	gen_ai_poc_databrickscoe	sdlc_wizard_customer	Parquet
Table	`s3://adif-sdlc/catalog/sdlc_wizard/order/`	gen_ai_poc_databrickscoe	sdlc_wizard_order	Parquet
Table + Glue Table	`s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`	gen_ai_poc_databrickscoe	ordersummary	Hudi (COPY_ON_WRITE)
Table	`s3://adif-sdlc/analytics/customeraggregatespend/`	gen_ai_poc_databrickscoe	customeraggregatespend	Parquet

Catalog Registration Expectations

- All target tables must be registered in Glue Data Catalog database `gen\_ai\_poc\_databrickscoe`
- Catalog registration should occur immediately after S3 write
- Use Glue API or Spark SQL CREATE TABLE for registration
- For Hudi tables, use Hudi's built-in Hive sync or manual Glue API registration
- Table metadata should include: location, format, schema, SerDe properties
- Ensure catalog tables are queryable via Amazon Athena after registration

Partitioning Expectations

- Customer Catalog Table: No partitioning

- Order Catalog Table: No partitioning
- Order Summary Hudi Table: No partitioning
- Customer Aggregate Spend Table: No partitioning

Note: Partitioning is not specified in FRD. If data volume grows significantly, consider partitioning by date or region in future iterations.

File Formats / Table Formats

Entity	Storage Format	Compression	SerDe	
-----	-----	-----	-----	
Customer Catalog	Parquet	Snappy (default)	Parquet SerDe	
Order Catalog	Parquet	Snappy (default)	Parquet SerDe	
Order Summary	Hudi (Parquet base)	Snappy (default)	Hudi SerDe	
Customer Aggregate Spend	Parquet	Snappy (default)	Parquet SerDe	

Update / Overwrite / Append Behavior

Entity	Write Mode	Behavior	Rationale
-----	-----	-----	-----
Customer Catalog	Overwrite	Replace all data on each run	Full refresh of catalog table
Order Catalog	Overwrite	Replace all data on each run	Full refresh of catalog table
Order Summary	Upsert (Hudi)	Insert new records, update existing based on record key	SCD Type 2 requires upsert to maintain history
Customer Aggregate Spend	Overwrite	Replace all data on each run	Full recalculation of aggregates

- Important Notes:
- Order Summary uses Hudi upsert operation to implement SCD Type 2 logic
- Upsert behavior: If OrderId exists, update with latest OpTs; if not, insert new record
- For SCD Type 2, "update" means inserting new active row and closing previous active row
- Customer Catalog and Order Catalog use overwrite to ensure clean state
- Customer Aggregate Spend uses overwrite for full recalculation (no incremental aggregation specified)
-

5. Processing Details

Data Quality Rules

ID	Rule Description	Applied To	Implementation
-----	-----	-----	-----
001	Remove rows with "Null" string literal (case-insensitive)	Customer, Order	Filter out rows where any column contains "null" (case-insensitive)

002	Remove rows with NULL or empty values	Customer, Order	Filter out rows where any column is NULL or empty string
003	Remove duplicate rows	Customer, Order	Use Spark dropDuplicates() on all columns

Deduplication Rules

- Method: Exact match deduplication across all columns
- Applied To: Customer and Order datasets after null removal
- Implementation: Spark DataFrame `dropDuplicates()` without parameters
- Behavior: Keep first occurrence, remove subsequent duplicates
- Logging: Log count of duplicates removed for each dataset

Join Rules

	Right Table	Join Type	Join Condition	Purpose
	-----	-----	-----	-----
	Order (cleaned)	Inner	Customer.CustId = Order.CustId	Create order summary
	Order (cleaned)	Inner	Customer.CustId = Order.CustId	Calculate customer metrics
Order Summary (IsActive=True)	New Order Summary	Left Outer	ExistingOrderSummary.OrderId = NewOrderSummary.OrderId	Identify changed customers

- Join Behavior:
- Inner joins exclude orders without matching customers
- Inner joins exclude customers without orders (for aggregation and order summary)
- Left outer join in SCD Type 2 logic identifies new orders (no match in existing table)

Aggregation Rules

ID	Source	Group By	Aggregation Function	Output Column	Purpose
	-----	-----	-----	-----	-----
	Customer-Order Join	Customer.Name, Order.Date	SUM(Order.PricePerUnit × Order.Qty)	TotalAmount	Calculate total spend per customer

- Aggregation Details:
- Calculate TotalAmount for each row before aggregation: `PricePerUnit Qty`
- Group by customer Name (not CustId) and order Date
- Sum TotalAmount within each group
- Result includes: Name, TotalAmount, Date

SCD / History Tracking Rules

- SCD Type 2 Implementation for Order Summary:

Attribute	Description	Implementation
-----	-----	-----

Record Key	OrderId	Unique identifier for each order
Change Detection Attributes	Customer.Name, Customer.EmailId, Customer.Region	Compare these attributes to detect customer changes
IsActive	Boolean flag indicating current record	True for active record, False for historical
StartDate	Timestamp when record became active	Set to job execution timestamp for new/updated records
EndDate	Timestamp when record became inactive	NULL for active records, job execution timestamp for closed records
Operation Timestamp	Operation timestamp	Job execution timestamp, used as Hudi precombine key

• SCD Type 2 Logic:

1. Initial Load: All records inserted with IsActive=True, StartDate=current_timestamp, EndDate=NULL

2. Incremental Load:

• Unchanged Records: No action (existing active record remains)

• Changed Customer Attributes:

• Close existing active record: Set IsActive=False, EndDate=current_timestamp, OpTs=current_timestamp

• Insert new active record: Set IsActive=True, StartDate=current_timestamp, EndDate=NULL, OpTs=current_timestamp

• New Orders: Insert with IsActive=True, StartDate=current_timestamp, EndDate=NULL, OpTs=current_timestamp

3. Change Detection: Compare Name, EmailId, Region between existing active records and new customer data

4. Hudi Upsert: Use OrderId as record key and OpTs as precombine key to ensure correct record versioning

Null Handling

Scenario	Handling Rule	Implementation	
-----	-----	-----	
"Null" string literal in source CSV	Remove row	Filter out rows containing "null" (case-insensitive) in any column	
NULL/empty value in source CSV	Remove row	Filter out rows with NULL or empty string in any column	
NULL in calculated TotalAmount	Log error, fail job	Validate that PricePerUnit and Qty are not NULL before calculation	
NULL in SCD Type 2 EndDate	Expected for active records	EndDate is NULL for IsActive=True records	
NULL in Hudi record key (OrderId)	Fail job	Validate OrderId is not NULL before Hudi write	

Type Casting Expectations

Source Column	Source Type	Target Column	Target Type	Casting Rule
-----	-----	-----	-----	-----
Customer.CustId	String (CSV)	Customer.CustId	String	Direct map, no casting
Customer.Name	String (CSV)	Customer.Name	String	Direct map, no casting
Customer.EmailId	String (CSV)	Customer.EmailId	String	Direct map, no casting

Customer.Region	String (CSV)	Customer.Region	String	Direct map, no casting
Order.OrderId	String (CSV)	Order.OrderId	String	Direct map, no casting
Order.ItemName	String (CSV)	Order.ItemName	String	Direct map, no casting
Order.PricePerUnit	String (CSV)	Order.PricePerUnit	Double	Cast to Double during schema definition
Order.Qty	String (CSV)	Order.Qty	Double	Cast to Double during schema definition
Order.Date	String (CSV)	Order.Date	String	Direct map, no casting (kept as String)
Order.CustId	String (CSV)	Order.CustId	String	Direct map, no casting
OrderSummary.IsActive	N/A	OrderSummary.IsActive	Boolean	Derived, set to True or False
OrderSummary.StartDate	N/A	OrderSummary.StartDate	Timestamp	Derived, set to current_timestamp()
OrderSummary.EndDate	N/A	OrderSummary.EndDate	Timestamp	Derived, set to NULL or current_timestamp()
OrderSummary.OpTs	N/A	OrderSummary.OpTs	Timestamp	Derived, set to current_timestamp()
Order.PricePerUnit × Order.Qty	Double	CustomerAggregateSpend.TotalAmount	Double	Calculated, no additional casting

- Type Casting Notes:
- CSV string values for PricePerUnit and Qty are cast to Double during schema definition at read time
- If casting fails (non-numeric values), Spark will produce NULL, which will be filtered out by data quality rules
- Date field is kept as String (no date parsing specified in FRD)
- Timestamp fields (StartDate, EndDate, OpTs) use Spark's `current\_timestamp()` function
- Boolean IsActive field is set programmatically (True/False literals)
-

6. Traceability Matrix

Input ID(s)	Original Source	Technical Source
---	-----	-----
	`s3://adif-sdlc/sdlc_wizard/customerdata/`	`s3://adif-sdlc/sdlc_wizard/customerdata/`
	`s3://adif-sdlc/sdlc_wizard/orderdata/`	`s3://adif-sdlc/sdlc_wizard/orderdata/`
	In-memory DataFrames from TR-INGEST-001, TR-INGEST-002	In-memory DataFrames from TR-INGEST-001, TR-INGEST-002
001	In-memory DataFrames from TR-CLEAN-001	In-memory DataFrames from TR-CLEAN-001
	In-memory DataFrames from TR-CLEAN-001	In-memory DataFrames from TR-CLEAN-001
	In-memory DataFrames from TR-CLEAN-001 + Existing Hudi table	In-memory DataFrames from TR-CLEAN-001 + Existing Hudi table
	In-memory DataFrames from TR-CLEAN-001	In-memory DataFrames from TR-CLEAN-001

- Source Fidelity Verification:
- All technical sources match original sources from FRD exactly
- No intermediate sources, staging tables, or alternative sources introduced
- In-memory DataFrames are direct results of processing original sources

- Hudi table source for incremental processing is the same table being updated (ordersummary)
-

7. ETL Mapping Requirements

TR-INGEST-001: Customer Data Ingestion Mapping

- Source: `s3://adif-sdlc/sdlc\_wizard/customerdata/` (CSV)
- Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Target: In-memory DataFrame `customer\_df`
- Target Schema: Same as source schema
- Column Mapping:

	Source Column	Target Column	Transformation	Notes	
	-----	-----	-----	-----	
	CustId	CustId	Direct map	No transformation	
	Name	Name	Direct map	No transformation	
	EmailId	EmailId	Direct map	No transformation	
	Region	Region	Direct map	No transformation	

-

TR-INGEST-002: Order Data Ingestion Mapping

- Source: `s3://adif-sdlc/sdlc\_wizard/orderdata/` (CSV)
- Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	

	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: In-memory DataFrame `order\_df`
- Target Schema: Same as source schema
- Column Mapping:

	Source Column	Target Column	Transformation	Notes	
	-----	-----	-----	-----	
	OrderId	OrderId	Direct map	No transformation	
	ItemName	ItemName	Direct map	No transformation	
	PricePerUnit	PricePerUnit	Direct map	Cast to Double at read time	
	Qty	Qty	Direct map	Cast to Double at read time	
	Date	Date	Direct map	No transformation	
	CustId	CustId	Direct map	No transformation	

•

TR-CLEAN-001: Data Quality Cleaning Mapping

- Source: In-memory DataFrames `customer\_df`, `order\_df`
- Target: In-memory DataFrames `customer\_cleaned\_df`, `order\_cleaned\_df`
- Column Mapping:
- All columns: Direct map (no schema change)
- Transformation: Row-level filtering (remove nulls and duplicates)
-

TR-CATALOG-001: Glue Catalog Registration Mapping

- Customer Catalog Mapping:
- Source: In-memory DataFrame `customer\_cleaned\_df`
- Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	

	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Target: Glue Table `gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_customer`
- Target Schema: Same as source schema
- Column Mapping:

	Source Column	Target Column	Transformation	Notes	
	-----	-----	-----	-----	
	CustId	CustId	Direct map	No transformation	
	Name	Name	Direct map	No transformation	
	EmailId	EmailId	Direct map	No transformation	
	Region	Region	Direct map	No transformation	

- Order Catalog Mapping:
- Source: In-memory DataFrame `order\_cleaned\_df`
- Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: Glue Table `gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_order`
- Target Schema: Same as source schema
- Column Mapping:

	Source Column	Target Column	Transformation	Notes	
	-----	-----	-----	-----	
	OrderId	OrderId	Direct map	No transformation	
	ItemName	ItemName	Direct map	No transformation	
	PricePerUnit	PricePerUnit	Direct map	No transformation	
	Qty	Qty	Direct map	No transformation	
	Date	Date	Direct map	No transformation	

	CustId	CustId	Direct map	No transformation	
--	--------	--------	------------	-------------------	--

•

TR-SCD2-001: Order Summary SCD Type 2 Mapping

- Source 1: In-memory DataFrame `customer\_cleaned\_df`

- Source 1 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Source 2: In-memory DataFrame `order\_cleaned\_df`

- Source 2 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: Hudi Table `gen\_ai\_poc\_databrickscoe.ordersummary`

- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	
	Name	String	Variable	No	

	EmailId	String	Variable	No	
	Region	String	Variable	No	
	IsActive	Boolean	Boolean	No	
	StartDate	Timestamp	Timestamp	No	
	EndDate	Timestamp	Timestamp	Yes	
	OpTs	Timestamp	Timestamp	No	

- Column Mapping:

Target Column	Source Column(s)	Transformation	Notes	
-----	-----	-----	-----	
OrderId	order_cleaned_df.OrderId	Direct map	From Order source	
ItemName	order_cleaned_df.ItemName	Direct map	From Order source	
PricePerUnit	order_cleaned_df.PricePerUnit	Direct map	From Order source	
Qty	order_cleaned_df.Qty	Direct map	From Order source	
Date	order_cleaned_df.Date	Direct map	From Order source	
CustId	order_cleaned_df.CustId	Direct map	From Order source (join key)	
Name	customer_cleaned_df.Name	Direct map	From Customer source via join on CustId	
EmailId	customer_cleaned_df.EmailId	Direct map	From Customer source via join on CustId	
Region	customer_cleaned_df.Region	Direct map	From Customer source via join on CustId	
IsActive	N/A	Derived	Set to True for active records, False for historical	
StartDate	N/A	Derived	Set to current_timestamp() for new/updated records	
EndDate	N/A	Derived	Set to NULL for active records, current_timestamp() for closed records	
OpTs	N/A	Derived	Set to current_timestamp() (Hudi precombine key)	

-

TR-AGG-001: Customer Aggregate Spend Mapping

- Source 1: In-memory DataFrame `customer\_cleaned\_df`

- Source 1 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Source 2: In-memory DataFrame `order\_cleaned\_df`

- Source 2 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Double	Double precision	No	
	Qty	Double	Double precision	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

- Target: Glue Table `gen\_ai\_poc\_databrickscoe.customeraggregatespend`
- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	Name	String	Variable	No	
	TotalAmount	Double	Double precision	No	
	Date	String	Variable	No	

- Column Mapping:

Target Column	Source Column(s)	Transformation	Notes
-----	-----	-----	-----
Name	customer_cleaned_df.Name	Direct map	From Customer source via join on CustId, used in GROUP BY
TotalAmount	order_cleaned_df.PricePerUnit, order_cleaned_df.Qty	Derived	SUM(PricePerUnit × Qty) grouped by Name and Date
Date	order_cleaned_df.Date	Direct map	From Order source, used in GROUP BY

- Aggregation Logic:
 1. Join customer_cleaned_df and order_cleaned_df on CustId
 2. Calculate row-level TotalAmount = PricePerUnit × Qty
 3. Group by customer_cleaned_df.Name and order_cleaned_df.Date
 4. Aggregate: SUM(TotalAmount) as TotalAmount
 5. Select: Name, TotalAmount, Date
-
- End of Technical Requirements Document